LAB 5

STATIC CODE ANALYSIS

Movie Theater Seat Reservation System

Student Name	Ridhima Jain
SRN	PES1UG24CS373
Course / Subject	Software Engineering
Assigned Repository	SETAPESU26/L5_09_movie_seats
Submission Date	30-10-2026

Consolidated report: Pylint • Bandit • Flake8

1. Introduction to Static Code Analysis

Static Code Analysis is the process of examining source code without executing the program. It helps detect potential defects, unsafe coding practices, maintainability concerns, and style violations early in the development process. This allows developers to improve code quality, readability, reliability, and security before runtime.

In this lab, the Movie Theater Seat Reservation System Python program was analysed using Pylint, Bandit, and Flake8. The original source code was analysed first, identified issues were corrected, and the tools were run again to verify the changes.

1.1 Tools used

Tool	Purpose	Checks performed
Pylint	General Python code-quality analysis.	Docstrings, naming conventions, unused variables, exception handling, code structure, and maintainability.
Bandit	Security-focused static analysis for Python.	Potentially unsafe coding patterns, weak cryptographic hashes, broad exception handling, and insecure file permissions.
Flake8	Python style and formatting checks.	PEP 8 violations, blank-line spacing, line length, whitespace, and selected programming errors.

1.2 Program analysed

The program manages movie-show seat reservations. Its functions allow users to reserve and release seats, generate ticket codes, save and load reservation data in JSON format, and display a seating summary. The analysis focused on improving code quality, security, and formatting while preserving the intended behaviour of the program.

2. Initial analysis — before fixing

The first analysis was performed on the original version of theater_system.py. The saved before reports document the issues detected before the corrective changes were applied. Since the original terminal screenshots are unavailable, the text reports are provided as the evidence for this stage.

3. Issues identified and corrective actions

The following table documents representative issues identified during the initial analysis. High- and medium-severity findings are listed before lower-severity style findings. Line numbers refer to the original source file before fixes.

Issue	Type	Line(s)	Description	Fix Approach
Weak MD5 hash used for ticket-code generation	Security — High (Bandit B324)	25	MD5 is considered cryptographically weak and is not recommended where a stronger hash is appropriate.	Replaced hashlib.md5() with hashlib.sha256().
Overly permissive file permissions	Security — High (Bandit B103)	55	The file was assigned permission mode 0o777, which grants overly broad access.	Changed the permission mode to 0o600.
Mutable default argument in reserve_seat()	Code Quality — Medium (Pylint W0102)	9	Using [] as a default argument can cause the same list object to be reused across function calls.	Changed the default to None and created a new list inside the function when required.
Bare except clause in release_seat()	Error Handling — Medium (Pylint W0702; Bandit B110; Flake8 E722)	21	A bare except can catch unexpected exceptions and hide the actual cause of an error.	Caught only the expected KeyError and ValueError exceptions.
Files opened without explicit encoding and context managers	Code Quality — Medium (Pylint W1514, R1732)	28, 33	Files were opened without an explicit encoding and without a context manager.	Used with open(..., encoding='utf-8') for consistent encoding and automatic file closure.

Missing module and function docstrings	Maintainability — Medium (Pylint C0114, C0116)	1, 9, 18, 24, 27, 31, 37, 42	The module and functions lacked documentation explaining their purpose.	Added a module-level docstring and function docstrings.
Non-standard function naming	Code Style — Low (Pylint C0103)	37	printSeatingSummary() did not follow Python's snake_case naming convention.	Renamed the function to print_seating_summary().
Incorrect blank-line spacing	Code Style — Low (Flake8 E302, E305)	9, 18, 24, 27, 31, 37, 42, 57	Function definitions did not have the required blank-line spacing.	Added the required blank lines between function definitions and after the final function.
Lines exceeding the configured length	Code Style — Low (Flake8 E501; Pylint C0301)	12, 40	Some source-code lines exceeded the configured maximum line length.	Wrapped long statements across multiple lines.

Note: The table summarises the main findings. The full tool reports in Section 5 should be treated as the authoritative record of the tool output.

4. Analysis after fixing

After the corrections were applied, the updated theater_system.py file was analysed again using all three tools. The saved after reports document the final verification results. The final Pylint score was 10/10, and Bandit reported no issues. Insert the complete Flake8 after report to document the final style check.

5. Before-and-after tool reports

The six text reports are reproduced in this section. Keep each report complete, including its summary, findings, counts, timestamps, and score where available. Where a placeholder appears, paste the exact contents of the corresponding .txt file.

5.1 Pylint — before

File: pylint_report_before.txt

theater_system.py | pylint_report_before.txt

pylint_report_before.txt

```
1  ************* Module theater_system
2  theater_system.py:40:0: C0301: Line too long (104/100) (line-too-long)
3  theater_system.py:1:0: C0114: Missing module docstring (missing-module-docstring)
4  theater_system.py:9:0: C0116: Missing function or method docstring (missing-function-docstring)
5  theater_system.py:9:0: W0102: Dangerous default value [] as argument (dangerous-default-value)
6  theater_system.py:12:27: C0209: Formatting a regular string which could be an f-string (consider-using-f-string)
7  theater_system.py:15:23: C0209: Formatting a regular string which could be an f-string (consider-using-f-string)
8  theater_system.py:18:0: C0116: Missing function or method docstring (missing-function-docstring)
9  theater_system.py:21:4: W0702: No exception type(s) specified (bare-except)
10 theater_system.py:24:0: C0116: Missing function or method docstring (missing-function-docstring)
11 theater_system.py:27:0: C0116: Missing function or method docstring (missing-function-docstring)
12 theater_system.py:28:8: W1514: Using open without explicitly specifying an encoding (unspecified-encoding)
13 theater_system.py:28:8: R1732: Consider using 'with' for resource-allocating operations (consider-using-with)
14 theater_system.py:31:0: C0116: Missing function or method docstring (missing-function-docstring)
15 theater_system.py:32:4: W0603: Using the global statement (global-statement)
16 theater_system.py:33:8: W1514: Using open without explicitly specifying an encoding (unspecified-encoding)
17 theater_system.py:33:8: R1732: Consider using 'with' for resource-allocating operations (consider-using-with)
18 theater_system.py:37:0: C0116: Missing function or method docstring (missing-function-docstring)
19 theater_system.py:37:0: C0103: Function name "printSeatingSummary" doesn't conform to snake_case naming style (invalid-name)
20 theater_system.py:42:0: C0116: Missing function or method docstring (missing-function-docstring)
21
22 ----------------------------------
23 Your code has been rated at 5.87/10
24
25
```

5.2 Pylint — after

File: pylint_report_after.txt

The final score achieved was 10/10. (It went to 5.87-> 9.80->10.00)

```
--------------------------------------------------------------------
Your code has been rated at 10.00/10 (previous run: 9.80/10, +0.20)
```

5.3 Bandit — before

File: bandit_report_before.txt

theater_system.py | bandit_report_before.txt U

bandit_report_before.txt

```
Run started:2026-10-30 03:10:17.823145+00:00

Test results:
>> Issue: [B110:try_except_pass] Try, Except, Pass detected.
   Severity: Low   Confidence: High
   CWE: CWE-703 (https://cwe.mitre.org/data/definitions/703.html)
   More Info: https://bandit.readthedocs.io/en/1.9.4/plugins/b110_try_except_pass.html
   Location: .\theater_system.py:21:4
20          shows[show_id]["booked"].remove(seat_number)
21      except:
22          pass
23

--------------------------------------------------
>> Issue: [B324:hashlib] Use of weak MD5 hash for security. Consider usedforsecurity=False
   Severity: High   Confidence: High
   CWE: CWE-327 (https://cwe.mitre.org/data/definitions/327.html)
   More Info: https://bandit.readthedocs.io/en/1.9.4/plugins/b324_hashlib.html
   Location: .\theater_system.py:25:11
24  def ticket_code(show_id, seat_number):
25      return hashlib.md5((show_id + str(seat_number)).encode()).hexdigest()[:8]
26

--------------------------------------------------
>> Issue: [B103:set_bad_file_permissions] Chmod setting a permissive mask 0o777 on file (seats.json).
   Severity: High   Confidence: High
   CWE: CWE-732 (https://cwe.mitre.org/data/definitions/732.html)
   More Info: https://bandit.readthedocs.io/en/1.9.4/plugins/b103_set_bad_file_permissions.html
   Location: .\theater_system.py:55:4
54      printSeatingSummary()
55      os.chmod("seats.json", 0o777)
56

--------------------------------------------------

Code scanned:
    Total lines of code: 47
    Total lines skipped (#nosec): 0
    Total potential issues skipped due to specifically being disabled (e.g., #nosec BXXX): 0

Run metrics:
    Total issues (by severity):
        Undefined: 0
        Low: 1
        Medium: 0
        High: 2
    Total issues (by confidence):
        Undefined: 0
        Low: 0
        Medium: 0
        High: 3
Files skipped (0):

```

5.4 Bandit — after

File: bandit_report_after.txt

Final Bandit output supplied after the security fixes.

theater_system.py | bandit_report_after.txt U

bandit_report_after.txt

```
Run started:2026-10-30 03:38:02.136636+00:00

Test results:
    No issues identified.

Code scanned:
    Total lines of code: 74
    Total lines skipped (#nosec): 0
    Total potential issues skipped due to specifically being disabled (e.g., #nosec BXXX): 0

Run metrics:
    Total issues (by severity):
        Undefined: 0
        Low: 0
        Medium: 0
        High: 0
    Total issues (by confidence):
        Undefined: 0
        Low: 0
        Medium: 0
        High: 0
Files skipped (0):

```

5.5 Flake8 — before

File: flake8_report_before.txt

theater_system.py | flake8_report_before.txt U

flake8_report_before.txt

```
theater_system.py:9:1: E302 expected 2 blank lines, found 1
theater_system.py:12:80: E501 line too long (83 > 79 characters)
theater_system.py:18:1: E302 expected 2 blank lines, found 1
theater_system.py:21:5: E722 do not use bare 'except'
theater_system.py:24:1: E302 expected 2 blank lines, found 1
theater_system.py:27:1: E302 expected 2 blank lines, found 1
theater_system.py:31:1: E302 expected 2 blank lines, found 1
theater_system.py:37:1: E302 expected 2 blank lines, found 1
theater_system.py:40:80: E501 line too long (104 > 79 characters)
theater_system.py:42:1: E302 expected 2 blank lines, found 1
theater_system.py:57:1: E305 expected 2 blank lines after class or function definition, found 1

```

5.6 Flake8 — after

File: flake8_report_after.txt

The file is empty because there are no violations, 'No Flake8 violations were reported.'

```
(venv) C:\Users\ridhi\L5_09_movie_seats>flake8 --count theater_system.py
0
```

SCREENSHOT OF RESULT AFTER

```
(venv) C:\Users\ridhi\L5_09_movie_seats>pylint theater_system.py

--------------------------------------------------------------------
Your code has been rated at 10.00/10 (previous run: 10.00/10, +0.00)


(venv) C:\Users\ridhi\L5_09_movie_seats>bandit -r theater_system.py
[main]  INFO    profile include tests: None
[main]  INFO    profile exclude tests: None
[main]  INFO    cli include tests: None
[main]  INFO    cli exclude tests: None
[main]  INFO    running on Python 3.12.7
Run started:2026-10-30 10:02:25.538288+00:00

Test results:
        No issues identified.

Code scanned:
        Total lines of code: 74
        Total lines skipped (#nosec): 0

Run metrics:
        Total issues (by severity):
                Undefined: 0
                Low: 0
                Medium: 0
                High: 0
        Total issues (by confidence):
                Undefined: 0
                Low: 0
                Medium: 0
                High: 0
Files skipped (0):

(venv) C:\Users\ridhi\L5_09_movie_seats>flake8 --count theater_system.py
0
```

6. Conclusion

Static code analysis was used to identify code-quality, security, and formatting issues in the Movie Theater Seat Reservation System without executing the program. Pylint helped improve readability and maintainability, Bandit identified potentially insecure coding patterns, and Flake8 highlighted style violations. After the corrections, the program achieved a Pylint score of 10/10 and Bandit reported no issues. The final Flake8 report is included as evidence of the formatting verification. The before-and-after text reports document the analysis process and the improvements made.